Documentation technique Bloc 2 — Backend Node.js

Dossier : `Budget_App/Backend/`

En une phrase : le serveur Node sert les pages HTML **et** une API. Chaque utilisateur ne voit **que ses** données.

Outils utilisés : Node.js 20 (sans Framework), MySQL, mot de passe chiffré (bcrypt), jeton de connexion (JWT) valable 7 jours. Base de données : `Backend_node`.

Un **mémo des sigles** se trouve à la fin de ce fichier.

---

1. Comment c'est organisé

```
Téléphone / navigateur
        |
        |  pages HTML  (login, accueil, cartes…)
        |  données JSON (liste des opérations, solde…)
        |
   fichier server.js   ← tout passe par ici
        |
        |-- handlers/     logique (transactions, cartes, stats…)
        |-- models/       règles de validation
        |-- utils/db.js   connexion MySQL
```

---

2. Ce que l'utilisateur peut faire

Compte : s'inscrire, se connecter, se déconnecter, modifier son profil, réinitialiser le mot de passe.

Budget : voir le solde, ajouter / modifier / supprimer des opérations et des catégories, voir des graphiques.

Moyens : gérer des cartes (4 derniers chiffres seulement), enregistrer des bénéficiaires, recharger le compte, simuler un transfert ou une facture.

Admin : se connecter et lister les comptes. Pas d'écran d'administration complet.

On ne fait pas (volontairement) : vrais paiements bancaires, connexion Facebook/Google, application mobile native.

Déroulement type

1. L'utilisateur s'inscrit ou se connecte.
2. Le serveur pose un cookie de connexion.
3. L'accueil affiche le **solde** (revenus moins dépenses) et les 5 dernières opérations.
4. Il peut ensuite : ajouter une dépense, une catégorie, une carte, recharger, transférer.
5. Après un transfert, si le solde est trop bas → message d'erreur, rien n'est écrit.

---

3. Modèle de données (les 5 tables)

Un utilisateur a plusieurs opérations, catégories, cartes et bénéficiaires. Une opération **peut** avoir une catégorie (ce n'est pas obligatoire).

```
Utilisateur  1 -------- *  Opérations (transactions)
Utilisateur  1 -------- *  Catégories
Utilisateur  1 -------- *  Cartes
Utilisateur  1 -------- *  Bénéficiaires
Catégorie    1 -------- *  Opérations   (optionnel)
```

Table	À quoi ça sert	Infos importantes
`users`	Comptes	email unique, mot de passe chiffré, rôle user ou

Table	À quoi ça sert	Infos importantes
		admin
`transactions`	Revenus et dépenses	titre, montant, type (revenu ou dépense), date
`categories`	Libellés perso	ex. « Courses », « Salaire » — à chaque user les siens
`cards`	Cartes enregistrées	nom, **4 derniers chiffres**, date d'expiration, marque
`recipients`	Destinataires	nom, téléphone ou IBAN

Règle du solde : on ne le stocke pas « à la main ». On le recalcule :

> solde = somme des revenus − somme des dépenses (pour cet utilisateur)

La colonne `balance` existe encore dans `users` mais on ne s'en sert pas (reste d'une ancienne version).

On ne stocke **jamais** le numéro de carte complet ni le CVV.

---

4. Étapes des actions importantes

Connexion

1. Le navigateur envoie l'email et le mot de passe.
2. Le serveur cherche cet email dans MySQL.
3. Il compare le mot de passe avec le hash enregistré.
4. Si ça ne correspond pas → erreur « Identifiants invalides ».
5. Si ça correspond → il crée un jeton de 7 jours, le met dans un cookie, et le

renvoie aussi en JSON (pour Postman).

Ajouter une dépense

1. Le navigateur envoie le titre, le montant, le type « dépense ».
2. Le serveur vérifie le jeton (qui est cet utilisateur ?).
3. Il enregistre la ligne **avec l'id de cet utilisateur** (pas un id envoyé par le client).
4. Au prochain affichage de l'accueil, le solde est recalculé et a baissé.

Supprimer une catégorie

1. Le serveur compte combien d'opérations utilisent cette catégorie.
2. S'il y en a au moins une → refus (« X transactions utilisent cette catégorie »).
3. Sinon → suppression.

---

5. Fonctionnalités, en français

Inscription — email pas déjà pris, mot de passe chiffré, rôle « user ».

Connexion / déconnexion — jeton 7 jours ; sans jeton, les pages HTML renvoient vers le login.

Accueil — nom, solde calculé, 5 dernières opérations. Les « points fidélité » = 1 % du solde (indicatif, pas un vrai programme).

Opérations — créer, lire, modifier, supprimer. Toujours filtrées sur l'utilisateur connecté.

Catégories — idem. On ne supprime pas une catégorie encore utilisée.

Cartes — le numéro complet est vérifié (algorithme de Luhn) puis on ne garde que les 4 derniers chiffres + la marque (Visa, Mastercard…).

Bénéficiaires — téléphone ou compte bancaire.

Recharge — crée un revenu intitulé « Recharge ».

Transfert — vérifie le solde ; crée une dépense. **Limite :** l'autre personne n'est pas vraiment créditée (simulation).

Stats — totaux, évolution jour / mois, camembert par catégorie.

Pour tester avec Postman : voir `Postman_API_QuickStart.md`.

---

6. Sécurité (ce qui est vraiment en place)

1. Mot de passe jamais en clair (bcrypt).
2. Cookie de connexion illisible en JavaScript (httpOnly).
3. Requêtes SQL avec des `?` (pas de collage de texte utilisateur dans le SQL).
4. L'identité vient du jeton, pas d'un champ caché du formulaire.
5. Carte : contrôle Luhn + stockage des 4 derniers chiffres seulement.

Limites à dire : le jeton est aussi dans le JSON (risque si volé via une faille JS) ; le reset mot de passe est en mémoire (perdu si on redémarre le serveur) ; mot de passe MySQL encore écrit dans le fichier local.

---

7. Lancer

```
cd Budget_App/Backend
npm install
npm start
```

Ouvrir http://localhost:3000 — MySQL doit tourner.

Mémo des sigles (Bloc 2)

Sigle / mot	Signification simple
API	Le serveur répond en données (JSON), pas seulement en pages HTML.
JSON	Format texte des données échangées (`{"solde": 100}`).
HTTP	Le protocole du web (GET = lire, POST = créer, PUT = modifier, DELETE = supprimer).
CRUD	Les 4 actions : Créer, Lire, Modifier, Supprimer.
JWT	Petit jeton signé qui prouve que vous êtes connecté (valable 7 jours ici).
cookie httpOnly	Fichier de session que JavaScript de la page ne peut pas lire (réduit un vol via XSS).
bcrypt	Méthode pour chiffrer (hasher) le mot de passe. On ne peut pas le relire.
MySQL	La base de données (tables `users`, `transactions`…).
requête préparée / `?`	On n'injecte pas le texte de l'utilisateur dans le SQL → protection contre l'injection SQL.
user_id	Numéro de l'utilisateur propriétaire de la ligne.
income / expense	Revenu / dépense.
last4	Les 4 derniers chiffres de la carte.
Luhn	Calcul qui vérifie qu'un numéro de carte est bien formé (pas qu'il

Sigle / mot	Signification simple
	existe vraiment).
PAN / CVV	Numéro complet de carte / code au dos — on ne les stocke pas.
MCD	Modèle conceptuel : les « familles » de données et leurs liens (1 user → plusieurs cartes).
MLD	Les tables SQL concrètes.
UML	Schémas standard (cas d'utilisation, classes, enchaînements).
P2P	Transfert d'une personne à une autre.
OTP	Code à usage unique (ici : simulé, en mémoire).
XSS	Attaque : coller du JavaScript malveillant dans une page.
401 / 403 / 409	Pas connecté / connecté mais pas le droit / email déjà pris.